INTELIGÊNCIA ARTIFICIAL · UNICAP · 2026.2

Miniprojeto: aprendizado por reforço com vários agentes usando a MARLlib

Material de estudo para a equipe — do zero à apresentação

> **Artigo base:** Hu, S. et al. *MARLlib: A Scalable and Efficient Library for Multi-agent Reinforcement Learning*. Journal of Machine Learning Research, v. 24, n. 315, p. 1–23, 2023.
> **Código:** github.com/Replicable-MARL/MARLlib
>
> **Em uma frase:** rodamos o que o artigo fez, comparamos nossos números com os dele e mostramos que dá para treinar ~42% mais rápido sem perder desempenho.

ANTES DE COMEÇAR

Como usar este material

Este documento foi escrito para quem nunca leu o artigo nem mexeu no projeto. Leia na ordem: cada parte usa o que veio antes.

Parte	O que você vai entender
1. Conceitos básicos	O que é aprendizado por reforço, o que muda quando há vários agentes, e os termos que vão aparecer.
2. O artigo	Qual problema ele resolve, o que propõe, como valida e quais resultados obteve.
3. Aplicações no mundo real	Onde esse tipo de técnica é usado fora da sala de aula.
4. O que nós fizemos	A reprodução do artigo: ambiente, algoritmos, configuração e resultados.
5. A nossa modificação	O que mudamos, como, por que funciona e o que muda no mundo real.
6. Itens do enunciado	Respostas prontas para cada pergunta do roteiro do professor.
7. Perguntas prováveis	O que o professor pode perguntar e como responder.
8. Glossário	Termos técnicos em uma linha.

A ideia do projeto em 3 passos (dica do professor)

1. **Executar o que está no artigo** — treinamos os algoritmos IPPO e MAPPO no ambiente *simple_spread*, como o artigo.
2. **Comparar** nossos resultados com a Tabela 4 do artigo.
3. **Propor uma mudança** — trocamos a forma como os agentes usam a rede neural (uma rede para todos × uma por agente) e medimos o efeito no desempenho e no tempo de treino.

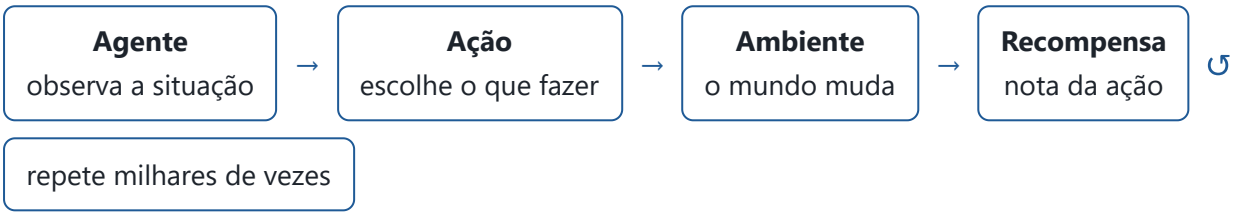
PARTE 1

Conceitos básicos

1.1 Aprendizado por reforço (RL)

É um jeito de ensinar um programa por **tentativa e erro**. Ninguém diz ao programa a resposta certa; ele tenta algo, recebe uma nota e vai ajustando o comportamento para conseguir notas melhores.

Analogia: adestrar um cachorro. Ele faz algo, ganha petisco (recompensa) ou não. Com o tempo ele repete o que dá petisco.



- **Agente:** quem aprende e age (um robô, um personagem de jogo).
- **Ambiente:** o "mundo" onde o agente está (uma simulação).
- **Observação:** o que o agente enxerga naquele momento.
- **Ação:** o que ele faz (ex.: mover para cima, baixo, esquerda, direita ou ficar parado).
- **Recompensa:** número que diz se aquilo foi bom ou ruim.
- **Episódio:** uma "partida" do começo ao fim. No nosso caso, 25 passos.
- **Política:** a "cabeça" do agente — a regra que transforma observação em ação. Aqui ela é uma **rede neural**.

> **Diferença para o aprendizado supervisionado**
>
> No supervisionado (ex.: classificar fotos de gatos) existe um conjunto de dados com as respostas certas, separado em treino e teste. No RL **não existe base de dados pronta**: os dados são gerados enquanto o agente joga. Por isso vários itens do enunciado (SMOTE, aumento de dados, validação cruzada) não se aplicam — ver Parte 6.

1.2 Vários agentes ao mesmo tempo (MARL)

MARL = *Multi-Agent Reinforcement Learning*, aprendizado por reforço multiagente. Vários agentes aprendem juntos no mesmo ambiente. Eles podem:

- **cooperar** (time de robôs num armazém — é o nosso caso),
- **competir** (dois times num jogo), ou
- **misto** (cooperam dentro do time e competem contra outro time).

O que fica difícil: enquanto um agente aprende, os outros também estão mudando. Para cada agente, o "mundo" fica instável, porque o comportamento dos colegas muda o tempo todo. Além disso, cada agente geralmente só enxerga uma parte do mundo (observação parcial).

1.3 PPO, o algoritmo de base

PPO (*Proximal Policy Optimization*) é um dos algoritmos de RL mais usados. Ele tem duas partes:

- **Ator** — a rede que decide a ação.
- **Crítico** — a rede que estima "quão boa é esta situação". Serve para o ator saber se uma ação foi melhor ou pior que o esperado.

O detalhe do PPO é que ele **muda a política aos poucos** (o "clipping" limita o tamanho de cada atualização). Isso deixa o treino estável.

1.4 Treino centralizado, execução descentralizada (CTDE)

Ideia central de muitos algoritmos multiagente: **durante o treino** pode-se usar informação de todos os agentes (como um técnico que vê o campo inteiro); **na hora de agir**, cada agente usa só o que ele mesmo enxerga (como o jogador em campo). Assim o treino fica mais bem informado, mas o sistema final continua funcionando sem comunicação central.

PARTE 2

O artigo: MARLlib

Título	MARLlib: A Scalable and Efficient Library for Multi-agent Reinforcement Learning
Autores	Siyi Hu, Yifan Zhong, Minquan Gao, Weixun Wang, Hao Dong, Zhihui Li, Xiaodan Liang, Xiaojun Chang, Yaodong Yang
Instituições	University of Technology Sydney, Peking University, NetEase, MBZUAI, entre outras
Periódico / ano	Journal of Machine Learning Research (JMLR), v. 24, n. 315, 2023 (publicado em agosto/2023)
Tipo de artigo	**Artigo de software**: não inventa um algoritmo novo; apresenta uma biblioteca (ferramenta) e mostra que ela funciona bem.

2.1 O problema

Quem pesquisa MARL enfrentava dois obstáculos:

1. **Cada ambiente fala uma "língua" diferente.** Um devolve a recompensa do time como um número só; outro dá uma recompensa para cada agente. Um fornece o estado global do mundo; outro não. Um identifica cada agente pelo nome; outro não. Para cada ambiente novo era preciso reescrever o código.
2. **Algoritmos e ambientes não combinam entre si.** Alguns algoritmos só funcionam em tarefas cooperativas; outros exigem o estado global. As bibliotecas da época (PyMARL, EPyMARL, MAPPO benchmark) cobriam poucos ambientes, quase sempre só cooperativos.

> **Analogia**
> Antes, cada aparelho tinha um carregador próprio. A MARLlib é como um padrão de tomada universal: qualquer ambiente se conecta a qualquer algoritmo.

2.2 A proposta: três mecanismos

① Adaptador padronizado de ambiente (*environment wrapper*)

Converte qualquer ambiente para o mesmo formato, em três passos: (1) coloca informações extras, como o estado global, dentro da observação; (2) copia a recompensa para cada agente, para todos terem a mesma estrutura; (3) etiqueta todos os dados com o identificador do agente.

② Fluxo de dados no nível do agente (*agent-level dataflow*)

Cada agente guarda os próprios dados e treina a si mesmo. Quando um algoritmo precisa de informação dos colegas, ela é trocada numa etapa intermediária (pós-processamento), antes do treino. Com isso, as três grandes famílias de algoritmos cabem no mesmo código:

Aprendizado independente (IL)
Cada agente aprende sozinho e trata os outros como parte do ambiente.
Ex.: IPPO, IQL.

Crítico centralizado (CC)
No treino, o crítico vê todos os agentes; na execução, cada um usa só a própria visão (CTDE).
Ex.: MAPPO, MADDPG.

Decomposição de valor (VD)
Aprende um valor para o time e divide entre os agentes.
Ex.: QMIX, VDN, VDPPO.

③ Estratégia de mapeamento de políticas (*policy mapping*)

Define **quais agentes usam a mesma rede neural**. Três opções prontas:

- **Compartilhamento total** (*full-sharing*): todos os agentes usam uma única rede.
- **Por grupo** (*group-sharing*): uma rede por time (ex.: uma para atacantes, outra para defensores).
- **Sem compartilhamento** (*non-sharing*): cada agente tem sua rede.

A biblioteca ainda **verifica automaticamente** se a combinação escolhida é válida — por exemplo, impede usar um algoritmo só cooperativo num jogo competitivo.

Guarde isto

O mecanismo ③ é o que a nossa modificação testa. O artigo oferece as opções de compartilhamento, mas **não mede** qual delas é melhor.

2.3 Como o artigo valida a biblioteca

- **Velocidade (Apêndice C):** treina o MAPPO num mapa do StarCraft (SMAC, mapa MMM2) por 1 milhão de passos, numa GPU RTX A6000. A MARLlib terminou em 1min24s–3min29s, contra 2min24s–5min29s das concorrentes. Em troca, usou mais memória.
- **Qualidade (Apêndice J):** roda 17 algoritmos em 23 tarefas de 5 ambientes (SMAC, MPE, Google Football, MAMuJoCo, MAgent), com 4 sementes aleatórias cada — mais de mil experimentos. No SMAC compara com a biblioteca EPyMARL: igual em 63% dos casos, melhor em 25%, um pouco pior em 12%.

Os números que usamos como referência vêm da Tabela 4 do artigo, para o ambiente MPE *simple_spread*:

Algoritmo	Família	Recompensa no artigo
IPPO	Independente	−65,74
MAPPO	Crítico centralizado	−66,26
IQL / VDN / QMIX / VDPPO	Independente / decomposição	−197,6 / −190,5 / −189,3 / −214,0

Quanto mais perto de zero, melhor. IPPO e MAPPO estão entre os melhores nesse cenário.

2.4 Conclusão do artigo

A MARLlib unifica ambientes e algoritmos num só lugar, é mais rápida que as alternativas e serve como plataforma de ensino e pesquisa. Como trabalho futuro, os autores citam **escalabilidade e eficiência**, explicabilidade e robustez (Apêndice K).

PARTE 3

Aplicações no mundo real

Sempre que há **várias máquinas ou programas tomando decisões ao mesmo tempo** e precisando se coordenar, MARL é candidato. Uma biblioteca como a MARLlib acelera a pesquisa nessas áreas porque permite testar muitos algoritmos sem reescrever código.

Área	Quem são os agentes	O que eles aprendem
Armazéns e logística	Robôs que buscam prateleiras (ex.: ambiente RWARE, suportado pela MARLlib)	Dividir pedidos e evitar congestionamento nos corredores
Drones e enxames	Cada drone	Cobrir uma área de busca e resgate sem colidir — é literalmente o que o *simple_spread* simula
Trânsito	Cada semáforo ou cada carro autônomo (ex.: MetaDrive, suportado)	Reduzir filas coordenando os cruzamentos
Energia elétrica	Inversores de painéis solares numa rede de distribuição (ex.: MAPDN, suportado)	Manter a tensão estável juntos
Jogos	Personagens controlados por IA (StarCraft — SMAC; futebol — Google Football)	Táticas de equipe; o AlphaStar da DeepMind chegou a nível de grão-mestre no StarCraft II
Indústria, finanças, saúde	Máquinas de uma linha de produção, robôs de negociação, sistemas de apoio	Citados pelo próprio artigo como direção futura (Apêndice K)

O cenário do nosso projeto, traduzido

No *simple_spread*, 3 agentes precisam se espalhar para cobrir 3 pontos sem bater uns nos outros. Pense em **3 drones que precisam se posicionar sobre 3 locais** de uma área de resgate, ou **3 robôs que precisam ocupar 3 estações** de trabalho. É um problema simples de cobertura e coordenação.

PARTE 4

O que nós fizemos: a reprodução

4.1 O ambiente: MPE *simple_spread*

- **MPE** (*Multi-agent Particle Environment*): um mundo 2D simples, com "partículas" (os agentes) e pontos de referência (marcos).
- **3 agentes e 3 marcos.** O time precisa cobrir todos os marcos.
- **Observação** de cada agente: sua posição e velocidade, e a posição relativa dos marcos e dos colegas.
- **Ações:** 5 opções (parado, cima, baixo, esquerda, direita).
- **Recompensa do time:** negativa — soma das distâncias de cada marco ao agente mais próximo, com penalidade por colisão. **Quanto mais perto de zero, melhor.** O valor reportado é a soma ao longo dos 25 passos do episódio.

4.2 Os algoritmos e por que eles

Escolhemos **IPPO** e **MAPPO**:

- **Cada um representa um lado da ideia central do artigo:** IPPO é aprendizado independente; MAPPO é crítico centralizado.
- **Comparação limpa:** os dois são PPO com os mesmos hiperparâmetros. A única diferença é que o crítico do MAPPO enxerga todos os agentes.
- **São os melhores nesse cenário segundo o próprio artigo** (−65,74 e −66,26), então temos um número claro para comparar.
- **Rodam em CPU comum.** Algoritmos como QMIX precisam de muito mais passos e, no artigo, foram muito piores nesse cenário.

4.3 Como foi executado

Item	Valor
Ferramentas	MARLlib (código oficial do artigo), Ray/RLlib 1.8, PyTorch 1.9, Python 3.8, Windows, só CPU
Rede neural	MLP (rede densa) com camadas de 128 e 256 neurônios
Hiperparâmetros	Os ajustados que a própria MARLlib fornece para o MPE: taxa de aprendizado 0,0005; clipping 0,3; bônus de entropia 0,01; 10 passadas por lote; lote de 128 episódios
Duração do treino	500 mil passos por execução (≈ 20 mil episódios)
Repetições	3 sementes aleatórias por configuração (o artigo usa 4)
Medida final	Média da recompensa nos últimos 10% do treino, depois média ± desvio padrão entre as 3 sementes
Total	12 execuções (2 algoritmos × 2 formas de compartilhamento × 3 sementes), ~4h30 de computador

O código cabe em quatro linhas de API da MARLlib:

```
env = marl.make_env(environment_name="mpe", map_name="simple_spread", force_coop=True)
algo = marl.algos.mappo(hyperparam_source="mpe")
model = marl.build_model(env, algo, {"core_arch": "mlp", "encode_layer": "128-256"})
algo.fit(env, model, stop={"timesteps_total": 500000}, share_policy="all", seed=0)
```

4.4 Resultado da reprodução

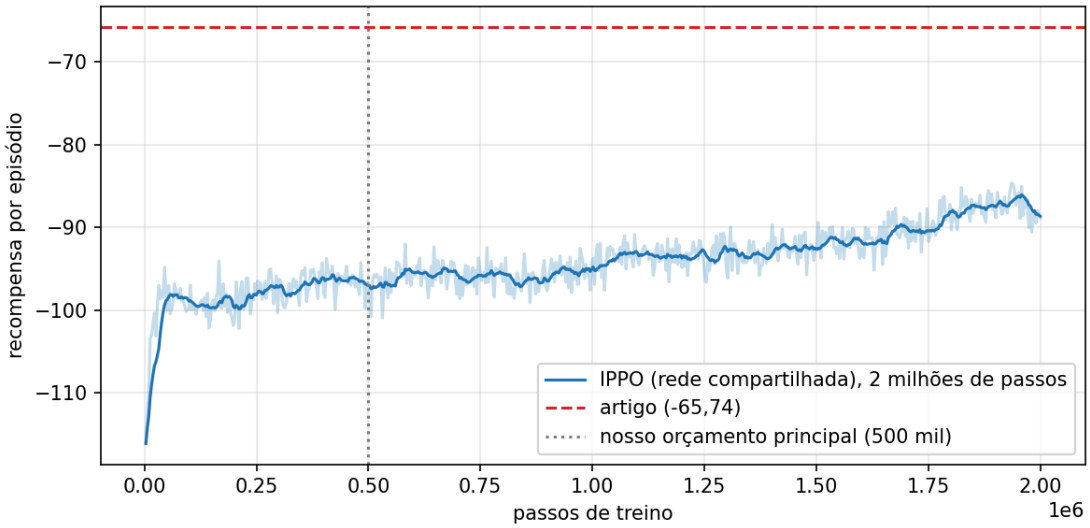
Algoritmo	Nosso resultado (rede compartilhada)	Artigo
IPPO	−96,4 ± 0,6	−65,74
MAPPO	−97,2 ± 0,8	−66,26

- **O que confirmamos:** IPPO e MAPPO empatam, como no artigo. Ou seja, nesse cenário simples, o crítico centralizado não traz vantagem.
- **O que não bateu:** nossos valores ficaram ~30 pontos abaixo dos do artigo.
- **Possíveis causas:** o artigo não diz quantos passos de treino usou no MPE (o padrão da MARLlib é 2 milhões; nós usamos 500 mil), e também não detalha a configuração exata do ambiente. Testamos a primeira hipótese com um treino 4× mais longo (seção 4.5): ajuda, mas não fecha a diferença.

4.5 Teste extra: treino 4× mais longo

Para testar se a diferença era falta de treino, rodamos o IPPO (rede compartilhada, semente 0) por **2 milhões de passos** — o padrão da MARLlib, 4× o nosso orçamento. Levou 66 min.

Ponto do treino	Recompensa (média da janela)
500 mil passos	−96,7
1 milhão	−95,3
1,5 milhão	−92,6
2 milhões (últimos 10% do treino)	**−87,5**
Artigo	−65,74



- **Treinar mais ajuda:** a recompensa continuou melhorando devagar e a distância para o artigo caiu de ~30 para ~22 pontos.
- **Mas não explica tudo:** mesmo com o orçamento padrão de 2 milhões ainda ficamos abaixo do artigo. O resto da diferença deve vir de detalhes que o artigo não publica (configuração exata do ambiente, número de passos real, forma de medir a recompensa).
- **Não muda a conclusão da modificação:** as duas formas de compartilhamento foram comparadas com o mesmo orçamento, então a comparação entre elas é justa.
- **Limite:** é uma única semente, então serve como indicação, não como resultado estatístico.

Como falar disso na apresentação

Não é um erro nosso: é um **achado sobre reprodutibilidade**. Usamos o código oficial e os hiperparâmetros oficiais e mesmo assim não chegamos ao número publicado. Isso acontece com frequência em RL e mostra que o artigo deveria informar a configuração completa de cada experimento.

PARTE 5

A nossa modificação

5.1 O que mudamos

Testamos a **estratégia de compartilhamento de rede** — o mecanismo ③ do artigo:

Rede compartilhada (padrão do artigo)
Os 3 agentes usam **a mesma rede neural**. Cada um recebe a sua própria observação, então eles ainda agem de forma diferente — mas o "cérebro" é um só.

Uma rede por agente (variação)
Cada agente tem **sua própria rede**, treinada só com a experiência dele. São 3 cérebros independentes.

Pergunta: dá para treinar mais rápido sem perder desempenho? Agentes iguais precisam de cérebros separados?

5.2 Por que essa modificação é válida (justificativa para o professor)

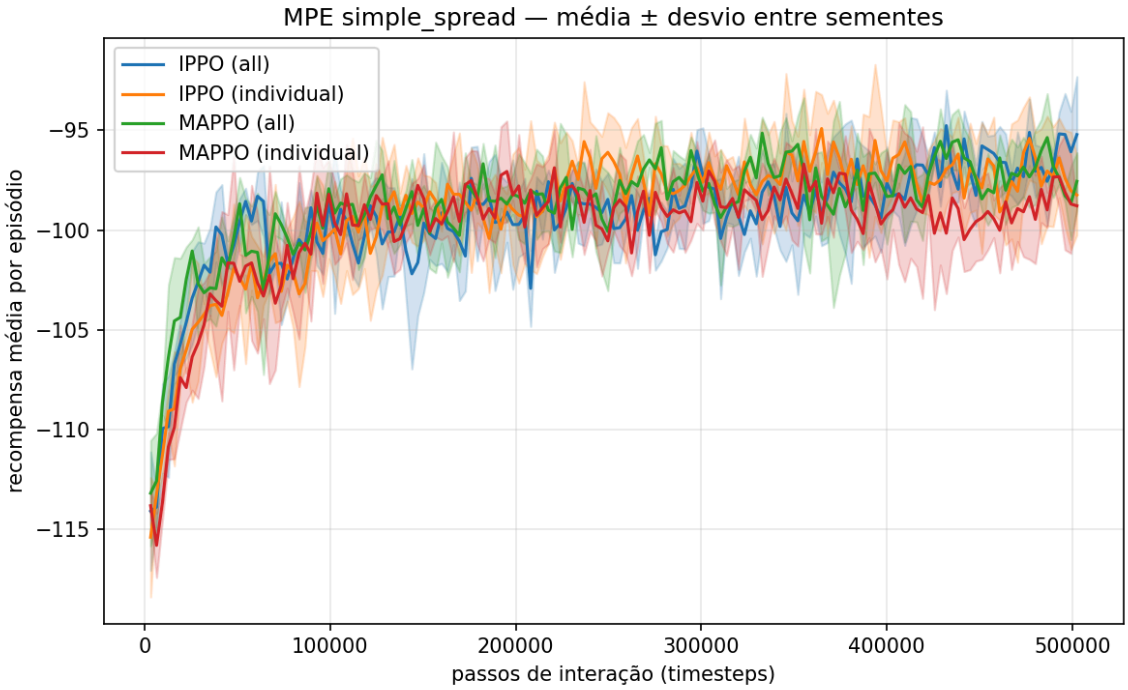
- O compartilhamento é **uma das três contribuições centrais** do artigo (Seção 2), mas o artigo **nunca mede o efeito dele**: no benchmark (Apêndice J) cada tarefa roda só com o padrão.
- O artigo diz que desliga o compartilhamento em tarefas com muitos agentes "para economizar memória" (Apêndice H.2), **sem quantificar**. Nós medimos o custo.
- Conversa com a alegação de **eficiência** do artigo (Apêndice C) e com o trabalho futuro que ele mesmo propõe: **escalabilidade e eficiência** (Apêndice K).

5.3 Como fizemos

Mudando **um único parâmetro** da MARLlib: `share_policy="all"` (rede compartilhada) ou `share_policy="individual"` (uma rede por agente). Todo o resto ficou idêntico: mesmo ambiente, mesmos hiperparâmetros, mesma rede, mesmos 500 mil passos, mesmas 3 sementes. Assim, qualquer diferença vem só do compartilhamento. Medimos a recompensa final e o tempo de treino.

5.4 Resultados

Algoritmo	Rede	Recompensa final	Tempo por execução
IPPO	compartilhada	**−96,4 ± 0,6**	**17,0 min**
IPPO	uma por agente	−97,4 ± 2,1	28,7 min
MAPPO	compartilhada	**−97,2 ± 0,8**	**17,8 min**
MAPPO	uma por agente	−98,7 ± 1,5	30,5 min



Curvas de aprendizado (média ± desvio entre 3 sementes). Eixo x: passos de treino. Eixo y: recompensa por episódio.

Conclusão da modificação

- Com a rede compartilhada, o treino ficou **~42% mais rápido** (17 contra 30 min).
- O desempenho ficou **igual ou um pouco melhor** (a diferença é pequena, da ordem de 1 a 1,5 ponto).
- Os resultados variaram **menos entre sementes** (desvio 0,6–0,8 contra 1,5–2,1): o treino é mais previsível.
- As curvas mostram que quase todo o aprendizado acontece nos primeiros ~100 mil passos; depois a melhora é lenta.

5.5 Por que funciona

1. **Menos trabalho por atualização.** Com uma rede só, o computador atualiza 1 rede por rodada. Com redes separadas, precisa atualizar 3 redes, cada uma com suas 10 passadas

sobre os dados. O total de experiência é o mesmo, mas o custo de processamento quase dobra.

2. **Cada rede aprende com 3× mais experiência.** A rede compartilhada recebe as jogadas dos 3 agentes; uma rede individual só vê as do seu agente. Mais exemplos por rede significa aprendizado mais estável — por isso o desvio entre sementes caiu.
3. **Os agentes são iguais.** No *simple_spread* os 3 agentes têm o mesmo papel, as mesmas ações e o mesmo objetivo. Não há nada que exija um "cérebro" diferente para cada um. O comportamento diferente vem das observações diferentes (cada um vê o mundo da sua posição).

> **Analogia**
>
> Três motoristas de entrega que fazem o mesmo trabalho podem seguir o **mesmo manual**, e cada um aplica o manual onde está. Escrever um manual separado para cada motorista dá três vezes mais trabalho e cada manual fica baseado em menos experiência.

5.6 O que muda nas aplicações do mundo real

- **Treino mais barato:** 42% menos tempo de computador significa menos custo de nuvem/GPU e mais experimentos no mesmo prazo.
- **Escala para muitos agentes:** com rede compartilhada, uma frota de 100 drones ou robôs iguais continua com **uma** rede, em vez de 100. Memória e tempo deixam de crescer com o número de agentes.
- **Agentes novos entram prontos:** um robô que chega ao armazém já usa a rede treinada, sem treinar do zero.
- **Quando NÃO usar:** se os agentes tiverem papéis diferentes (ex.: drone de câmera e drone de carga, atacante e goleiro), uma rede só pode não servir. Aí o mais indicado é o compartilhamento **por grupo** (uma rede por tipo de agente), opção que a MARLlib também oferece.

> **Limites (diga antes que perguntem)**
>
> Testamos um único cenário, com agentes iguais, 3 sementes e em CPU. Em tarefas com agentes diferentes o resultado pode ser outro. O ganho de tempo é medido no nosso computador; o valor exato muda com o hardware, mas a tendência (1 rede é mais barata que 3) não.

PARTE 6

Itens do enunciado — respostas prontas

Item	Resposta
Artigo	MARLlib, Hu et al., Journal of Machine Learning Research, v. 24, 2023 (dentro de 2022–2026).
Base de dados	Não há base fixa: os dados são gerados pela interação dos agentes com o ambiente MPE *simple_spread* (3 agentes, 3 marcos, 25 passos por episódio, 5 ações).
Tamanho da população	Não se aplica (não é inteligência de enxame). São 3 agentes.
% treino / teste	Não se aplica no formato supervisionado. A avaliação usa a recompensa nos episódios dos últimos 10% do treino.
Validação cruzada	Não. No lugar, repetimos cada configuração com 3 sementes aleatórias e reportamos média ± desvio.
Algoritmos	IPPO (independente) e MAPPO (crítico centralizado), ambos baseados em PPO.
Hiperparâmetros	Os ajustados que a MARLlib fornece para o MPE. Sem busca em grade, para manter a comparação com o artigo.
Transfer learning	Não.
Regularização / dropout	Bônus de entropia (0,01) e clipping do PPO (0,3). Sem dropout.
Aumento de dados	Não se aplica.
Desbalanceamento (SMOTE etc.)	Não se aplica (não é um problema de classificação).
Figuras de mérito	Recompensa média por episódio, curva de aprendizado, desvio entre sementes e tempo de treino.
Implementação e fontes	Código oficial da MARLlib (GitHub + documentação), Ray/RLlib, PyTorch; scripts próprios para rodar e analisar.
Modificação (2,0 pts)	Avaliação da estratégia de compartilhamento de rede (compartilhada × uma por agente), não medida no artigo: ~42% menos tempo sem perda de desempenho.

PARTE 7

Perguntas prováveis e como responder

Por que o resultado de vocês é pior que o do artigo?

Usamos o código e os hiperparâmetros oficiais. O artigo não informa quantos passos usou no MPE nem todos os detalhes do ambiente. Com um treino 4× mais longo (2 milhões de passos, o padrão da biblioteca) a diferença caiu de ~30 para ~22 pontos (seção 4.5): parte é falta de treino, o resto vem de detalhes não publicados. A diferença aparece nas duas configurações igualmente, então não afeta a comparação da nossa modificação.

Por que não usaram validação cruzada ou divisão treino/teste?

Porque em aprendizado por reforço não existe uma base de dados fixa para dividir. O equivalente é repetir o experimento com sementes aleatórias diferentes, que é o que o artigo também faz (4 sementes; nós usamos 3).

Se os 3 agentes usam a mesma rede, por que não fazem todos a mesma coisa?

Porque cada um recebe uma observação diferente — a posição dele e o que está ao redor dele. A mesma regra aplicada em lugares diferentes gera ações diferentes.

Por que IPPO e MAPPO deram praticamente o mesmo resultado?

O artigo também mostra isso (−65,7 contra −66,3). No *simple_spread* cada agente já enxerga os colegas e os marcos, então o crítico centralizado do MAPPO tem pouca informação extra a acrescentar.

Por que só esse ambiente e esses algoritmos?

É o cenário cooperativo clássico, usado pelo artigo e roda sem instalações pesadas. Os outros (StarCraft, MuJoCo, Football) exigem Linux, GPU ou softwares externos. IPPO e MAPPO são os melhores nele, segundo o artigo, e formam uma comparação limpa entre duas famílias.

A diferença de recompensa entre compartilhada e individual é significativa?

É pequena (1 a 1,5 ponto), da ordem do desvio. A conclusão que defendemos é "**não perde** desempenho", não "ganha muito". O ganho claro é no tempo (~42%) e na estabilidade (menor desvio).

Por que o treino com redes individuais é mais lento se a quantidade de dados é a mesma?

Porque são 3 redes para atualizar em vez de 1, cada uma com suas próprias passadas de otimização. O custo de processamento cresce com o número de redes.

Isso funcionaria com agentes diferentes?

Provavelmente não tão bem. Nesse caso o indicado é compartilhar por grupo (uma rede por tipo de agente). Testar isso seria um bom trabalho futuro.

O que é uma semente aleatória?

Um número que define os sorteios do treino (pesos iniciais da rede, posições iniciais no ambiente). Sementes diferentes geram treinos diferentes; repetir com várias mostra se o

resultado é consistente ou foi sorte.

O que significa uma recompensa de −96?

É a soma das penalidades ao longo dos 25 passos de um episódio: distância dos marcos ao agente mais próximo mais colisões. Zero seria perfeito (todos os marcos cobertos desde o primeiro passo, sem colisões), o que é impossível porque os agentes começam longe.

PARTE 8

Glossário

Termo	Em uma linha
Agente	Quem observa, age e aprende.
Ambiente	A simulação onde os agentes agem.
Recompensa	Nota numérica que indica se a ação foi boa.
Episódio	Uma partida completa (25 passos no nosso caso).
Passo (timestep)	Um instante em que cada agente escolhe uma ação.
Política	A regra (rede neural) que escolhe a ação a partir da observação.
Ator / crítico	Rede que decide / rede que avalia a situação.
PPO	Algoritmo de RL que melhora a política em passos pequenos e seguros.
IPPO	PPO em que cada agente aprende de forma independente.
MAPPO	PPO com crítico centralizado, que vê todos os agentes no treino.
CTDE	Treino com informação de todos, execução com informação local.
MARL	Aprendizado por reforço com vários agentes.
MPE	Ambiente 2D simples de partículas, muito usado em MARL.
Compartilhamento de política	Vários agentes usando a mesma rede neural.
Semente (seed)	Número que fixa os sorteios de um treino.
Hiperparâmetro	Configuração escolhida antes do treino (ex.: taxa de aprendizado).
Ray / RLlib	Bibliotecas de computação distribuída e de RL sobre as quais a MARLlib é construída.